
Draft-and-Audit Reinforcement Learning for Optimization Modeling

Zeping Min¹ Weihang Xu^{1 2} Zhengzhong You¹ Wotao Yin¹ Xinshang Wang¹

Abstract

Natural language to optimization (NL2Opt) requires translating unstructured text into executable mathematical models. Beyond simple syntax errors, this task suffers from *silent modeling failures*, where incorrect formulations execute successfully but yield invalid results. We propose **Draft-and-Audit RL (DA-RL)**, a framework that learns optimization modeling as a two-step iterative workflow. Unlike inference-time scaffolds that rely on intermediate solver feedback to guide repairs, DA-RL optimizes a shared-parameter policy using terminal-only verification: the model is rewarded solely based on the execution of the final audited program. This constraint forces the model to internalize rubric-guided revision as a learned capability and encourages the emergence of *cross-turn synergy*, where the policy learns to generate drafts that are structurally amenable to self-correction.

1. Introduction

Optimization modeling translates real-world decision problems across logistics (Toth & Vigo, 2002) and scheduling (Pinedo, 2012) into mathematical programs with decision variables, constraints, and objectives. Large language models (LLMs) have demonstrated strong capabilities in natural language understanding, reasoning, and code generation across an expanding range of applications (Schick et al., 2023; Parisi et al., 2022; Lewkowycz et al., 2022). Building on these capabilities, recent work has explored *natural language to optimization* (NL2Opt) (Ramamonjison et al., 2023): users describe a problem in text, and an LLM produces a solver-ready formulation that can be directly executed. Despite promising results, NL2Opt remains challenging because the generated artifact must be simultaneously (i) semantically faithful to the problem statement, (ii) syn-

tactically and API-correct (e.g., for PySCIPOpt), and (iii) numerically consistent. Crucially, failures are not limited to crashes: small logical or numerical mistakes can yield *silent modeling failures* where the solver returns a plausible but incorrect numeric answer, which is particularly dangerous for downstream decision making.

Most existing NL2Opt approaches fall into two broad families. First, prompting and agentic scaffolds (Wei et al., 2023; Yao et al., 2023; Shinn et al., 2023; Madaan et al., 2023) keep the base model frozen while structuring generation into stages (e.g., extraction $\rightarrow$ formulation $\rightarrow$ code), sometimes with iterative refinement (AhmadiTeshnizi et al., 2024). These methods can improve correctness by externalizing an engineering workflow into prompts and orchestration, but the resulting behavior is often sensitive to prompt design and inference-time control logic, and may not persist when the scaffold is altered or removed. Second, training-based adaptation via supervised fine-tuning (Ouyang et al., 2022; Wei et al., 2022) or execution-aware reinforcement learning (Schulman et al., 2017; Le et al., 2022) can improve modeling knowledge and code style (see Sec. 2). However, many training formulations still optimize a *single-pass* final program, with iterative improvement delegated to inference-time loops.

We propose to learn NL2Opt as a **short interactive workflow** rather than a one-shot translation. Human modelers rarely produce a correct formulation in a single attempt; instead, they draft a model and then audit it against a small checklist (e.g., constraint completeness, parameter consistency, and variable domains). We train this rubric-guided revision behavior directly into the policy parameters. We optimize a strict two-turn *Draft* $\rightarrow$ *Audit* policy with **shared parameters** across turns, under a **terminal-only verification** interface: the solver is invoked *exactly once* on the audited output, and the model receives *no intermediate tool feedback* (e.g., solver traces) between turns. This differs from prompting-based draft-revise loops, where multi-step behavior is imposed by orchestration, and from execution-aware training that optimizes only a single final program while delegating iterative improvement to inference-time refinement. Because reward is observed only after executing the audited program, training can also encourage *audit-friendly drafting*: first-pass solutions that are internally consistent and easier to verify and repair under the

¹Alibaba DAMO Academy ²University of Washington. Correspondence to: Zeping Min <minzeping.mzp@alibaba-inc.com>.

same rubric, which we refer to as *cross-turn synergy*.

Figure 1 summarizes DA-RL: a strict two-turn Draft→Audit interaction with a *single* terminal execution verifier. The draft is never executed and the model receives no solver traces between turns, so revision must be learned from terminal correctness rather than reactive tool feedback.

Our contributions are:

- **Draft→Audit policy for NL2Opt:** We formulate NL2Opt as a two-step policy that first drafts and then audits/revises a solver program under a fixed rubric.
- **Learned rubric-guided revision:** The audit rubric defines a stable interface for targeted checks (constraints, parameters, variable domains), and training internalizes this behavior into model parameters rather than relying on inference-time scaffolds.
- **Empirical evidence for cross-turn synergy:** Controlled ablations show that DA-RL outperforms matched single-turn RL and that its Turn-1 drafts are already stronger, indicating that terminal reward propagates useful pressure to both drafting and auditing.

Conflict of Interest Disclosure. The authors Zeping Min, Weihang Xu, Zhengzhong You, Wotao Yin, and Xinshang Wang were employed by Alibaba DAMO Academy during this work. Alibaba leads the development of the Qwen model family, which was among the model families evaluated in this paper and used as the backbone for DA-RL.

2. Related Work

2.1. Natural Language to Optimization Modeling

NL2Opt studies how to translate natural-language problem descriptions into executable optimization models (Ramanonjison et al., 2023). Prior work establishes this setting as a structured generation task where end-to-end execution is essential: a program can execute successfully while still encoding an incorrect formulation. Our work targets this silent-failure mode through a learned Draft→Audit policy trained under terminal execution-based verification.

2.2. Prompting and Agentic Scaffolds for Structured Generation

A common approach to NL2Opt is to keep the base LLM fixed and impose structure at inference time via prompting or orchestration, such as staged generation (e.g., extraction → formulation → code) and iterative refinement loops (Wei et al., 2023; AhmadiTeshnizi et al., 2024). More broadly, chain-of-thought and self-reflection methods use explicit intermediate reasoning and revision to improve multi-step

tasks (Wei et al., 2023; Shinn et al., 2023). These scaffolds can boost correctness by enforcing an engineered workflow, but their behavior may depend on prompt design and control logic and does not necessarily persist when the scaffold is altered or removed. OptiMUS, for example, uses solver execution and debugging feedback inside an inference-time agent loop (AhmadiTeshnizi et al., 2024). This style is useful for execution failures, but solver traces are less informative for silent modeling failures where code runs and returns a plausible but wrong objective. In contrast, we optimize a fixed two-turn interface and train the model to apply an audit rubric without intermediate tool feedback.

2.3. Training-Based Adaptation and Execution-Aware Learning

Another line of work improves NL2Opt by adapting model parameters through supervised fine-tuning and solver-aware training. LLMOpt proposes training intended to generalize across diverse optimization problem types (Jiang et al., 2024), while ORLM emphasizes domain adaptation and instruction tuning for optimization modeling and solver-code generation (Huang et al., 2025). Complementary to supervised learning, execution-based reinforcement learning can align generations with sandboxed correctness signals (Le et al., 2022; Chen et al., 2025). Most prior training setups, however, optimize the correctness of a *single* generated program, with iterative improvement handled externally (e.g., test-time refinement). SIRL uses solver-informed rewards for optimization modeling, but still trains single-turn solver-code generation (Chen et al., 2025). Our approach instead trains a two-step Draft→Audit *policy* and uses *terminal-only verification* by executing only the final audited program.

3. Background and Problem Formulation

We focus on the translation of natural language specifications into executable Mixed-Integer Linear Programs (MILPs).

3.1. Mixed-Integer Linear Programming (MILP)

A standard MILP formulation seeks to minimize a linear objective function subject to linear inequality constraints and integrality requirements on a subset of variables. Formally, let $z \in \mathbb{R}^n$ be the vector of decision variables. A generic MILP instance is defined by parameters $(c, A, b, \mathcal{I})$:

$$\begin{aligned} \min_z \quad & c^\top z \\ \text{s.t.} \quad & Az \leq b \\ & z_j \in \mathbb{Z}, \quad \forall j \in \mathcal{I} \subseteq \{1, \dots, n\} \end{aligned} \tag{1}$$

where $c \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$, and $b \in \mathbb{R}^m$. The set $\mathcal{I}$ denotes the indices of variables constrained to be integers. In the NL2Opt setting, the challenge is not solving Eq. (1) given

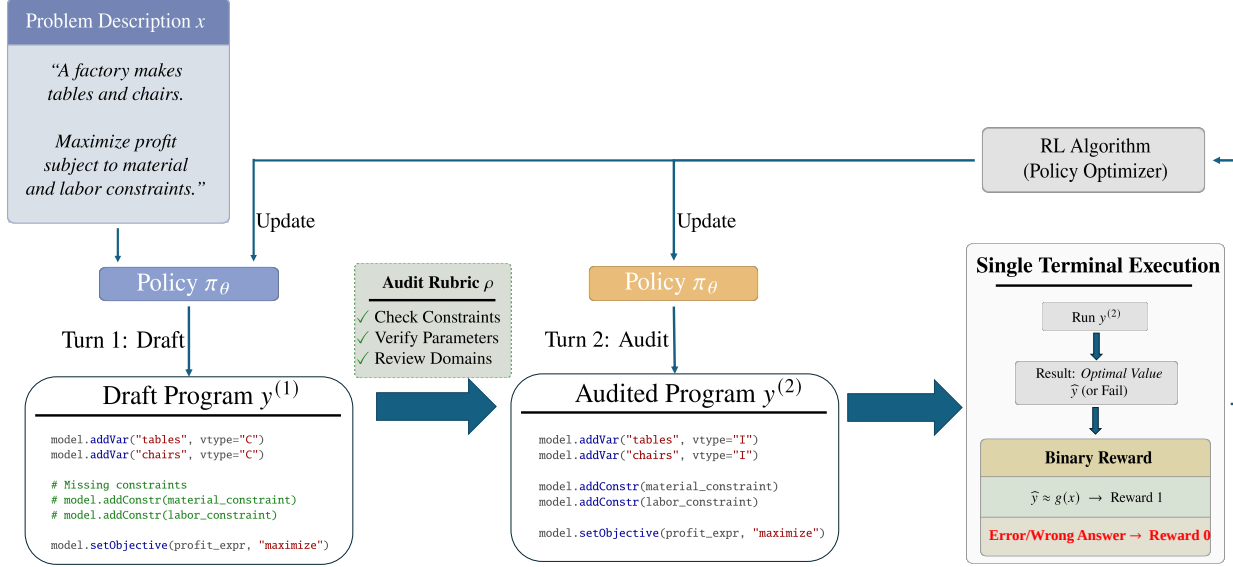


Figure 1. **DA-RL overview (Draft→Audit with terminal-only verification)**. Given a word problem x , the policy first generates a draft program $y^{(1)}$ (Turn 1). It then audits and optionally revises the draft under a fixed rubric ρ to produce a final program $y^{(2)}$ (Turn 2). **Single terminal execution verifier**: only $y^{(2)}$ is executed to compute the binary reward, with no intermediate tool feedback between turns.

the parameters, but rather correctly instantiating the symbolic parameters (c, A, b, T) and their associated variable domains entirely from unstructured text.

3.2. The NL2Opt Task

Let $\mathcal{X}$ denote the space of natural-language optimization problems and $\mathcal{Y}$ denote the space of executable solver programs (e.g., Python code using PySCIPOpt). An instance is a pair $(x, g(x))$, where $x \in \mathcal{X}$ is the word problem and $g(x) \in \mathbb{R}$ is the ground-truth optimal objective value. The goal is to learn a parameterized generator $\pi_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ that maps x to a program $y \in \mathcal{Y}$ which (i) executes successfully and (ii) encodes a mathematically correct model for the problem described by x .

3.3. Evaluation via Execution

Because equivalent optimization models admit many algebraic realizations, we evaluate correctness by executing the generated program in a sandbox and comparing its numeric output to the ground truth (Ramamonjison et al., 2023; Le et al., 2022). Let $\mathcal{E} : \mathcal{Y} \rightarrow \mathbb{R} \cup \{\perp\}$ denote sandboxed execution of a candidate program: $\mathcal{E}(y)$ is the solver’s returned optimal objective value if execution succeeds, and $\mathcal{E}(y) = \perp$ if the program fails (e.g., syntax/runtime errors or no valid output). We define a binary terminal verifier using the tolerance rule in our evaluation protocol:

$$r(x, y) = \mathbb{I}[\mathcal{E}(y) \neq \perp \wedge \text{Match}(\mathcal{E}(y), g(x); \tau)], \quad (2)$$

where $\text{Match}(\cdot)$ is the dual absolute/relative tolerance rule specified in Appendix C. This signal penalizes both execution failures and *silent modeling failures*, where code runs but returns an incorrect numeric answer (Turpin et al., 2023).

Execution-based evaluation is necessary because many mathematically equivalent formulations exist, while small formulation mistakes may either crash execution or (more subtly) produce a plausible but incorrect objective value, i.e., a silent modeling failure.

4. Methodology

Our goal is to train an LLM that converts a natural-language optimization problem x into an executable PySCIPOpt program. We treat NL2Opt as a short two-turn workflow: the model first drafts a complete solution, then audits and revises it using a fixed rubric.

4.1. Draft-and-Audit Protocol

We implement a strict two-turn interaction protocol where the model first generates a complete draft, then performs a structured audit.

Turn 1: Drafting. Given a problem description x , the policy generates a draft program $y^{(1)}$ intended as a complete PySCIPOpt solution. The draft turn receives problem text, variable-type guidelines (e.g., continuous vs. integer vari-

ables for different problem entities), and, when provided, dataset-specific few-shot exemplars that demonstrate correct API usage. The model is expected to produce syntactically valid code that attempts to capture all constraints, parameters, and the objective function described in x .

Turn 2: Auditing. After drafting, the model enters an audit phase. It receives the original problem x , the draft $y^{(1)}$, and a fixed audit rubric ρ that directs the model to check the code for three common sources of silent modeling failures:

1. whether constraints are consistent with the problem statement and whether any constraints are missing;
2. whether the numerical values of all parameters match the problem description;
3. whether variable types are appropriate (`vttype="C"` vs. `vttype="I"`).

The audit turn then produces a revised program $y^{(2)}$, which may be identical to $y^{(1)}$ if no corrections are deemed necessary.

Terminal-Only Verification. We use a **single terminal execution verifier**: we execute *only* the final audited program $y^{(2)}$ to compute reward, and provide *no intermediate tool feedback* (e.g., solver error traces or execution outcomes) between the two turns. The draft $y^{(1)}$ is never executed during training; it serves solely as an intermediate artifact for the audit turn to inspect. This design makes revision depend on patterns in terminal correctness rather than reactive debugging guided by environment messages.

4.2. Terminal Reward and Execution-Based Verification

To evaluate the correctness of a generated program, we rely on sandboxed execution and numerical comparison. Let $\mathcal{E} : \mathcal{Y} \rightarrow \mathbb{R} \cup \{\perp\}$ denote a sandbox execution function: given a candidate program y , $\mathcal{E}(y)$ returns the solver’s optimal objective value if execution succeeds, and $\mathcal{E}(y) = \perp$ if the program fails (e.g., due to syntax errors, runtime exceptions, or inability to parse a numeric answer). Given the ground-truth optimal value $g(x)$ for problem x and a tolerance threshold $\tau > 0$, we define a binary terminal verifier:

$$r(x, y) = \mathbb{1}[\mathcal{E}(y) \neq \perp \wedge \text{Match}(\mathcal{E}(y), g(x); \tau)]. \quad (3)$$

Here $\text{Match}(\cdot)$ uses the dual absolute/relative tolerance rule detailed in Appendix C. This verifier penalizes both execution failures and *silent modeling failures*—cases where code runs and returns a plausible number that is nonetheless incorrect due to formulation bugs.

Importantly, we compute $r(x, y)$ *only* on the audited program $y^{(2)}$; the draft $y^{(1)}$ is never executed. This terminal-only verification prevents the model from relying on inter-

mediate solver feedback to guide its revisions, so the audit step is based on the problem statement and the draft code.

4.3. Learning Objective and Optimization

For each problem instance x , the policy generates a two-turn trajectory with shared parameters:

$$y^{(1)} \sim \pi_\theta(\cdot | x), \quad y^{(2)} \sim \pi_\theta(\cdot | x, y^{(1)}, \rho), \quad (4)$$

where $y^{(1)}$ is the draft program and $y^{(2)}$ is the audited program under rubric ρ . Our objective maximizes the expected terminal correctness of the audited output:

$$\mathcal{J}(\theta) = \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y^{(1)} \sim \pi_\theta(\cdot | x)} \mathbb{E}_{y^{(2)} \sim \pi_\theta(\cdot | x, y^{(1)}, \rho)} \left[r(x, y^{(2)}) \right]. \quad (5)$$

We optimize $\mathcal{J}(\theta)$ using Proximal Policy Optimization (PPO) with ratio clipping. Because the reward is nonnegative and observed only after Turn 2, successful trajectories provide positive reinforcement to both (i) rubric-guided auditing behavior in Turn 2 and (ii) drafting strategies in Turn 1 that produce programs amenable to rubric-based correction.

4.4. Why Two Turns Help: Learned Revision and Cross-Turn Synergy

The two-turn structure with terminal-only verification provides two complementary benefits.

Learned Rubric-Guided Revision. The audit rubric ρ defines a checklist—constraint completeness, parameter consistency, and variable domains—that directs the model’s attention to common failure modes. Unlike inference-time scaffolds that parse solver error messages and attempt reactive fixes, our rubric is independent of execution outcomes and can be applied without running the draft. Training under terminal-only verification encourages the model to internalize this rubric: it must learn to identify violations (e.g., missing constraints, incorrect variable types) by inspecting the draft code and cross-referencing the problem statement, rather than relying on environment feedback. Over repeated training episodes, the audit turn learns a policy for conservative, targeted edits that address rubric violations without introducing new bugs.

Audit-Friendly Drafting via Shared Parameters. Because both turns share parameters θ and the reward depends only on the audited output $y^{(2)}$, optimization couples drafting and auditing. A draft that contains serious errors difficult to repair under the rubric (e.g., hallucinated constraints not mentioned in x , or structurally malformed code) will likely yield $r(x, y^{(2)}) = 0$ even after auditing. Consequently, successful trajectories tend to be those where the draft is already internally consistent and amenable to rubric-based verification. This incentive can reshape the distribution of drafts:

the model learns to produce first-turn outputs that encode the problem structure more accurately and avoid errors that are hard to detect or fix via the rubric alone. We refer to this coupling as *cross-turn synergy*—shared parameters and terminal-only verification encourage audit-friendly drafting, reducing silent modeling failures even before the audit turn is applied.

4.5. Prompts and the Role of Few-Shot Exemplars

Prompts define the two-turn interface and provide structural guidance during training.

Draft Prompt. We show the draft prompt in Appendix D.1. The draft prompt includes task instructions, output format specifications (e.g., code blocks with a final `print` statement), and variable-type guidelines that clarify when to use continuous vs. integer variables based on problem entities (e.g., counts of discrete items should use integers). For most datasets, we also supply dataset-specific few-shot exemplars—typically one or two worked examples showing correct formulations for representative problems from the same domain. These exemplars provide syntactic structure (e.g., API conventions) and concrete templates for encoding constraints and objectives; MAMO-Easy uses only the general variable-type tips because its instances are relatively direct.

Audit Prompt. We show the audit prompt in Appendix D.2. The audit prompt instantiates the rubric ρ by listing the three verification checks (constraints, parameters, domains) and enforces a conservative editing rule: the model should modify the draft *only if clear errors are identified*. This instruction discourages unnecessary rewrites that might break working code or introduce new bugs, encouraging the model to intervene selectively. The prompts structure the interaction and provide initial guidance. The fixed component is the audit interface; the learned component is the policy’s ability to apply the rubric reliably and produce audit-friendly drafts without relying on intermediate solver feedback.

5. Experiments

5.1. Experimental Setup

Benchmarks. We evaluate on five optimization modeling benchmarks: **NL4Opt** (Ramamonjison et al., 2023), **MAMO-Easy** and **MAMO-Complex** (Huang et al., 2024), **IndustryOR** (Huang et al., 2025), and **OptMATH-Bench** (Lu et al., 2025). Dataset sizes and additional descriptions are in Appendix B.

Public-Benchmark Split Protocol. Our main results use a *public-benchmark* training and evaluation regime. For

each dataset, we construct **three independent random 80/20 splits**, train only on the **80%** subset of the *public benchmark*, and evaluate on the held-out **20%** subset. This protocol measures *in-benchmark generalization*. Closed-API baselines are reported as point-in-time references under the same held-out splits.

Comparability Note. Some prior SFT/RL numbers are obtained under materially different supervision and/or data access than our public-split protocol (Table 1). For example, ORLM is seeded from real-world industrial cases described as existing in private settings, and then expanded via an instruction-synthesis pipeline. LLMOpt uses benchmark datasets as *seed data* for augmentation (with a small held-out subset for testing), rather than training strictly on an independently constructed public 80/20 split. Other works (e.g., OptMATH) train on large external synthetic corpora released separately from the benchmarks. We therefore mark such results as *reference-only* ([†]) in Table 2, and focus our main comparisons on methods evaluated under our public 80/20 split protocol.

Models and Baselines. Under our split protocol, we evaluate **prompting-only** baselines directly on the held-out test split (no benchmark fine-tuning), including closed-API models. Unless otherwise stated, prompting baselines use the same Draft→Audit inference protocol and execution pipeline as DA-RL, so their comparison isolates parameter training rather than prompt access.

Training (Ours). We train DA-RL from **Qwen2.5-7B-Instruct** using a binary terminal reward computed by executing only the audited program (Turn 2). Hyperparameters are detailed in Appendix A.

Evaluation. We report pass@1 execution accuracy: a solution is correct if the executed objective value matches ground truth under the dual tolerance rule (Appendix C).

Protocol Details. For consistency across models, our protocol fixes (i) the split construction (three 80/20 seeds per dataset), (ii) the execution verifier and tolerance (Appendix C), and (iii) the prompt templates including the audit rubric (Appendix D). Unless otherwise stated, all reported rows use this shared evaluation setup.

5.2. Main Results

Table 2 summarizes pass@1 accuracy across the five benchmarks.

DA-RL is trained on the corresponding 80% training split, while prompting-only baselines are evaluated without benchmark fine-tuning. Rows marked [†] are quoted from prior

Table 1. Training-data regimes of our model vs. [†] reference-only baselines. **Public** refers to whether the *training corpus* is fully publicly released. **Benchmark used** indicates whether benchmark instances are used for seeding/training (beyond evaluation).

Work	Public	Train size	Benchmark used
ORLM (Huang et al., 2025)	Partial	30k	No
LLMOpt (Jiang et al., 2024)	Partial	29.8k	Yes
OptMATH (Lu et al., 2025)	Yes	>200k	No
SIRL (Chen et al., 2025)	Partial	10k	No
Ours (DA-RL)	Yes	<2k	Yes

work under different training/evaluation access and are provided for context rather than direct comparison.

Among rows evaluated under the public-split protocol, **DA-RL (7B) achieves the highest Macro AVG (68.1%)**, suggesting that a learned Draft→Audit policy can substantially improve end-to-end correctness with terminal-only verification.

The gains are most pronounced on harder structured benchmarks: **MAMO-Complex reaches 84.5%**, indicating that rubric-guided auditing is effective at repairing constraint and domain errors that otherwise lead to silent modeling failures.

Effect of the Audit Prompt on Frontier LLMs. To verify that closed-API baselines receive the same two-turn opportunity as DA-RL, we also evaluate them under both single-turn (1T) and Draft→Audit (2T) prompting. Table 3 reports macro averages under the identical held-out splits and execution verifier. The audit turn improves all frontier LLMs, but the size of the gain varies widely: +5.6 pp for GPT-5.2, +8.0/+9.6 pp for Claude Sonnet/Opus, and +33.7 pp for Gemini-3-Pro. DA-RL differs because the two-turn behavior is trained into a 7B policy rather than imposed only at inference time. It matches the best frontier macro average (68.1 vs. 67.1) while exhibiting a distinct strength profile: much higher MAMO-Complex accuracy, but lower OptMATH-Bench accuracy.

5.3. Controlled Ablations: Training vs. Test-Time Auditing

Table 4 isolates the effect of learning a two-turn policy from simply adding an audit prompt at inference time. The no-RL base model gains only +2.4/+0.0 pp from Draft→Audit prompting on MAMO-Complex/IndustryOR, and applying the same audit prompt to a matched single-turn RL model adds only +2.4/+0.0 pp. In contrast, DA-RL improves over matched single-turn RL by +7.8/+16.7 pp. Moreover, DA-RL’s Turn-1 drafts already exceed single-turn RL by +3.9/+15.0 pp, although Turn 1 receives neither an audit prompt nor solver feedback at inference time. This is consistent with the cross-turn synergy hypothesis: terminal reward on the audited program reshapes both the auditing behavior and the distribution of drafts. A matched token-budget

control for single-turn RL ($2\times$ generation length) remains at 76.7/20.0, suggesting that longer responses alone do not explain the gain. Taken together, these contrasts suggest that DA-RL improves auditability upstream, before the audit turn is executed. We next report rubric and turn-count controls to test whether the result is specific to the structured two-turn design.

Rubric and Turn-Count Controls. We further test whether DA-RL’s gains depend on the structured audit rubric and on the two-turn horizon (Table 5). Replacing the structured checklist with a generic “check your code” audit prompt reduces accuracy by -5.4 pp on MAMO-Complex and -16.7 pp on IndustryOR, indicating that the rubric contributes more than a generic instruction to revise. Extending the rollout to three turns also degrades performance (-10.1/-21.7 pp), likely because the sparse terminal reward is spread over a longer trajectory and the additional revision turn increases opportunities for format drift or unnecessary edits. Thus, the effective design is not simply “more self-refinement”, but a short, structured audit trained with terminal-only verification. These controls point to two limits of naive self-refinement: unstructured audits lack a stable error taxonomy, and longer horizons make the terminal signal less attributable to a specific corrective action.

Transfer and Residual Failures. To probe whether DA-RL only memorizes within-benchmark patterns, we train a model exclusively on MAMO-Complex (169 training instances) and evaluate it on four unseen benchmarks without fine-tuning. This OOD model improves over the single-turn base model on MAMO-Easy (81.7→89.3), NL4Opt (32.6→71.7), IndustryOR (5.0→35.0), and OptMATH-Bench (2.9→11.8). Transfer is strongest for benchmarks sharing MILP-style modeling patterns and weakest on OptMATH-Bench, whose mathematical notation and coefficient extraction differ more from MAMO-Complex. Some residual errors are not modeling errors: a post-hoc diagnostic on the main DA-RL checkpoint shows that enforcing a single code block and increasing the response-length cap improves accuracy from 84.5% to 88.4% on MAMO-Complex and from 30.4% to 32.4% on OptMATH-Bench. We keep the main evaluation on the common strict extractor for comparability, but this diagnostic indicates that part of the residual gap is due to output-format compliance rather

Table 2. Performance on optimization modeling benchmarks (pass@1, %). **Public-split protocol (ours)**: train on **80%** of each *public* benchmark and evaluate on the held-out **20%**, averaged over three random splits. **Reference-only** rows ([†]) are quoted from prior work trained with different supervision and/or evaluated under different access (e.g., private-data training and/or full-benchmark evaluation), and are included for context rather than direct comparison. Bold entries mark the best result among rows evaluated under our public-split protocol.

Type	Model	Acc (pass@1), %					Macro AVG
		NL4Opt	MAMO-Easy	MAMO-Complex	IndustryOR	OptMATH-Bench	
<i>Reference-only: reported under different supervision and/or evaluation access (e.g., additional training corpora and/or different splits) [†]</i>							
Offline [†]	ORLM-LLaMA-3-8B	85.7	82.3	37.4	24.0	2.6	46.4
Offline [†]	LLMOpt-Qwen2.5-14B	80.3	89.5	44.1	29.0	12.5	51.1
Offline [†]	OptMATH-Qwen2.5-7B	94.7	86.5	51.2	20.0	24.4	55.4
Offline [†]	OptMATH-Qwen2.5-32B	95.9	89.9	54.1	31.0	34.7	61.1
Online-RL [†]	SIRL-Qwen2.5-7B	96.3	91.7	51.7	33.0	30.5	60.6
<i>Public-split protocol: evaluated on held-out 20% (mean over 3 random 80/20 splits)</i>							
Prompting	Claude-Sonnet-4.5	92.0	92.4	50.4	45.0	55.9	67.1
Prompting	Claude-Opus-4.5	90.6	91.1	56.6	40.0	56.9	67.0
Prompting	GPT-5.2	7.2	90.6	51.2	10.0	26.5	37.1
Prompting	Gemini-3-Pro	83.3	62.6	28.7	30.0	5.9	42.1
Ours	Draft-and-Audit RL (7B)	97.1	91.8	84.5	36.7	30.4	68.1

Table 3. Macro pass@1 (%) for frontier LLMs under single-turn and Draft→Audit prompting.

Model	1T	2T	Δ
Claude-Sonnet-4.5	59.1	67.1	+8.0
Claude-Opus-4.5	57.4	67.0	+9.6
GPT-5.2	31.5	37.1	+5.6
Gemini-3-Pro	8.4	42.1	+33.7
DA-RL (7B)	—	68.1	—

Table 4. Controlled ablations on two challenging benchmarks (pass@1, %). All rows use the same 7B backbone, data splits, terminal verifier, and prompt examples where applicable unless noted.

Setting	MAMO-C	IndustryOR
Base 1T (no RL)	30.2	5.0
Base 2T prompt (no RL)	32.6	5.0
Single-turn RL	76.7	20.0
Single-turn RL + audit prompt	79.1	20.0
DA-RL Turn-1 draft	80.6	35.0
DA-RL Turn-2 audit	84.5	36.7

than failed optimization modeling. We therefore view the reported numbers as conservative estimates under a strict code-only interface, rather than as an upper bound on the learned policy’s modeling ability.

Base-Model Audit Without RL. We additionally run the base Qwen2.5-7B-Instruct under the same Draft→Audit prompt to separate the prompt interface from reinforcement learning. Across all five benchmarks, the base model’s audit turn provides only a small macro gain (30.5→31.9), while DA-RL reaches 68.1 under the same evaluation pipeline. Per-benchmark improvements from RL over the base two-turn prompt are large on NL4Opt (+62.3 pp), MAMO-Complex (+51.9 pp), IndustryOR (+31.7 pp), and

Table 5. Protocol controls on two challenging benchmarks (pass@1, %). Deltas are relative to two-turn DA-RL with the structured rubric.

Setting	MAMO-C	Δ	IndustryOR	Δ
Structured rubric, 2 turns	84.5	—	36.7	—
Generic audit prompt	79.1	-5.4	20.0	-16.7
Three turns	74.4	-10.1	15.0	-21.7

OptMATH-Bench (+27.5 pp), with a smaller but still positive gain on MAMO-Easy (+7.8 pp). These results indicate that the two-turn prompt alone is not sufficient; effective auditing is learned by optimizing the two-turn policy with terminal rewards. Put differently, DA-RL learns a selective repair policy that preserves plausible drafts while intervening on likely modeling errors, rather than merely executing a generic reviewer instruction.

5.4. Ablation Study: Role of Few-Shot Examples

Finally, we examine the role of the few-shot exemplars used in most dataset prompts. To separate reinforcement learning from this prompt context, we trained a variant of DA-RL using **zero-shot prompts**, removing the input-output coding examples provided in the standard setting.

Performance Degradation. Removing few-shot examples causes a sharp performance drop, particularly on harder benchmarks. On MAMO-Complex, pass@1 accuracy decreases from 84.5% to 76.7%, while on the more challenging OptMATH-Bench, accuracy falls from 30.4% to 11.8%. The model still generates valid Python syntax, reflecting the instruction-tuned base model, but the large drop on OptMATH-Bench indicates weaker grounding from complex, unstructured problem descriptions to mathematical formulations without explicit templates.

Qualitative Failure Analysis. We analyzed the logs of the zero-shot trained model and identified two primary failure modes that few-shot examples typically prevent:

1. **Logical formulation reversals.** Without reference examples for advanced optimization patterns (e.g., maximizing the minimum margin), the zero-shot model often formulates the constraints backward. In the *Securities* problem (MAMO-Complex), the goal was to maximize the worst-case revenue. The zero-shot model defined a variable R and maximized it, but set the constraint as $R \geq \text{Revenue}_{\text{scenario}}$ (requiring revenue to be *at least* the scenario payoff) rather than $R \leq \text{Revenue}_{\text{scenario}}$. This makes the problem unbounded or logically incorrect, whereas the few-shot model infers the correct epigraph form ($z \leq f(x)$) from similar examples.
2. **Missing-data hallucinations.** Optimization problems often present data in unstructured, verbose lists. The zero-shot model frequently hallucinated that necessary parameters were missing. For instance, in the *Cell Tower* problem (OptMATH-Bench), the text listed coverage mappings explicitly (e.g., “Area 0 can be covered if any of the towers 1, 5... are built”). The zero-shot auditor outputs: “*The specific combinations of towers... are not provided in the problem statement, so we would need to define these,*” and generates placeholder code. Few-shot examples appear crucial not just for syntax, but for demonstrating the *intent* that the model should parse and restructure messy text data rather than waiting for a clean matrix input.

These results suggest that RL improves *verification*, while few-shot examples remain important for *grounding* the reasoning process in the specific format of optimization modeling.

5.5. Audit Behavior and Failure Modes

We next analyze diagnostic held-out logs from MAMO-Complex and OptMATH-Bench. This analysis characterizes how DA-RL improves performance and identifies remaining failure modes; it is separate from the three-split averages reported in Table 2.

5.5.1. HOW AUDITING REPAIRS DRAFTS: NET LIFT AND RECOVERY RATE

To characterize the auditor’s efficacy independent of the base model’s absolute performance, we evaluate paired outcomes in these diagnostic logs: for each instance we execute both the Turn-1 draft $y^{(1)}$ and the Turn-2 audited program $y^{(2)}$ under identical settings and record the outcome transition. From these transitions, we report **Net Lift** (the absolute

accuracy change on the diagnostic held-out logs),

$$\text{NetLift} = \frac{\#(\text{Fail} \rightarrow \text{Correct}) - \#(\text{Correct} \rightarrow \text{Fail})}{N},$$

Error Recovery Rate (the fraction of failed drafts that are repaired),

$$\text{Recovery} = \frac{\#(\text{Fail} \rightarrow \text{Correct})}{\#(\text{Fail in Turn-1})},$$

and a **Regression Rate** that quantifies the potential cost of auditing (how often the audit degrades an already-correct draft),

$$\text{Regress} = \frac{\#(\text{Correct} \rightarrow \text{Fail})}{\#(\text{Correct in Turn-1})}.$$

Figure 2 reports Net Lift and Recovery Rate; we compute Regression Rate from the same paired transition accounting. On MAMO-Complex, the auditor attains a diagnostic net lift of 14.0% with a recovery rate of 42.9%, while exhibiting no observed regressions (regression rate 0% on the analyzed logs). On OptMATH-Bench, the auditor yields a positive diagnostic net lift of 8.8% with a lower recovery rate of 10.7%, again with no observed regressions (regression rate 0%).

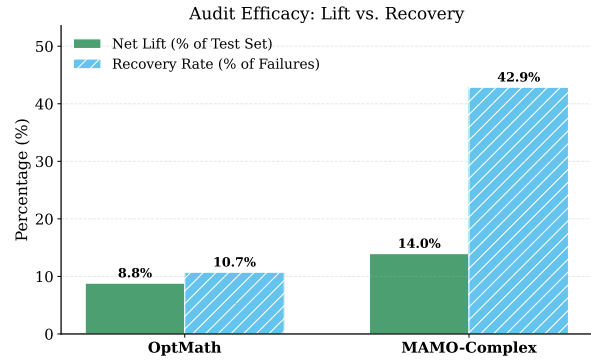


Figure 2. Audit efficacy on diagnostic held-out logs. **Net Lift** is the absolute accuracy change after auditing, and **Recovery Rate** is the fraction of failed drafts corrected by the audit step.

Interpretation. The joint view of lift, recovery, and regression separates improvement from risk. MAMO-Complex shows high recovery, consistent with rubric-aligned constraint-logic mistakes that can be repaired by code inspection, whereas OptMATH-Bench shows lower recovery under brittle domain-to-symbol mapping. Across both domains, the absence of regressions indicates that auditing acts as a conservative repair operator, delivering gains primarily through targeted Fail→Correct fixes.

5.5.2. FAILURE-MODE BREAKDOWN: EXECUTION VS. SILENT MODELING FAILURES

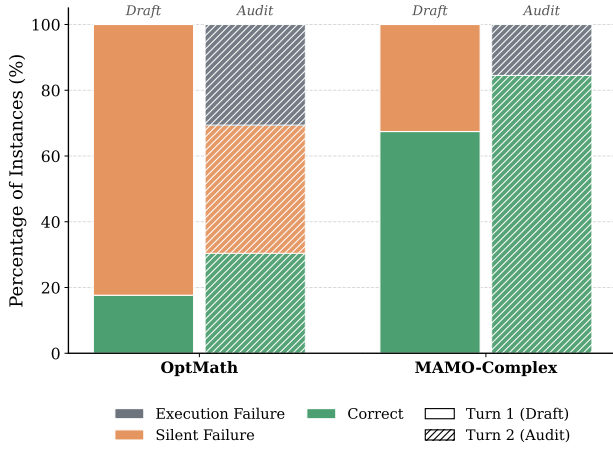


Figure 3. Outcome breakdown for draft and audited programs on diagnostic held-out logs. Auditing reduces silent modeling failures, while remaining execution failures are often associated with strict output-format requirements.

Auditing converts *Silent Modeling Failures* into correct solutions (Figure 3): on MAMO-Complex, silent failures drop from 32.6% in the draft to 0.0% after auditing. The main residual issue is *Execution Failure* (18.6% on MAMO-Complex, 32.4% on OptMATH-Bench), often caused by “Format Drift,” where the auditor identifies the logic bug but wraps corrected code in prose rather than a clean code block.

Audit Edits. We report the compact edit-frequency table in Appendix B.2. In the diagnostic logs, Turn-1/Turn-2 diffs show broad re-emission plus targeted semantic repair: constraints and domains are touched in all analyzed outputs, objective edits are frequent, and format edits remain common.

6. Conclusion

We presented Draft-and-Audit RL, a training approach for NL2Opt that treats optimization modeling as a short interactive workflow rather than a single-pass translation. By optimizing a two-turn policy under terminal-only verification, the model learns rubric-guided revision and produces audit-friendly drafts through cross-turn synergy. Our empirical evaluation demonstrates substantial improvements in end-to-end correctness, particularly on harder structured benchmarks where silent modeling failures are prevalent. The learned policy acts as a conservative repair operator, delivering accuracy gains through targeted fixes. Our analysis shows high recovery rates on constraint-logic errors and effective conversion of silent failures into correct solutions,

supporting the role of rubric-guided auditing in addressing key failure modes in optimization code generation.

These findings suggest that embedding verification workflows directly into policy training can yield more robust and data-efficient learning for structured code generation. The controlled ablations further show that the gains come from training the workflow itself, not from a longer response budget or an inference-time audit prompt alone. More broadly, this work demonstrates that treating complex generation tasks as learned multi-turn policies, rather than relying solely on single-pass translation or inference-time orchestration, offers a promising direction for improving reliability in domain-specific code synthesis.

Acknowledgements

We thank the reviewers and area chairs for their constructive feedback.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- AhmadiTeshnizi, A., Gao, W., and Udell, M. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *ICML’24*, pp. 577–596, Vienna, Austria, July 2024. JMLR.org.
- Chen, Y., Xia, J., Shao, S., Ge, D., and Ye, Y. Solver-Informed RL: Grounding Large Language Models for Authentic Optimization Modeling, 2025. URL <https://arxiv.org/abs/2505.11792>.
- Huang, C., Tang, Z., Hu, S., Jiang, R., Zheng, X., Ge, D., Wang, B., and Wang, Z. ORLM: A Customizable Framework in Training Large Models for Automated Optimization Modeling. *Operations Research*, 73(6):2986–3009, November 2025. ISSN 0030-364X. doi: 10.1287/opre.2024.1233. URL <https://pubsonline.informs.org/doi/abs/10.1287/opre.2024.1233>.
- Huang, X., Shen, Q., Hu, Y., Gao, A., and Wang, B. LLMs for Mathematical Modeling: Towards Bridging the Gap between Natural and Mathematical Languages, 2024. URL <https://arxiv.org/abs/2405.13144>.
- Jiang, C., Shu, X., Qian, H., Lu, X., Zhou, J., Zhou, A., and Yu, Y. LLMOPT: Learning to Define and Solve

- General Optimization Problems from Scratch, 2024. URL <https://arxiv.org/abs/2410.13213>.
- Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. C. CodeRL: Mastering code generation through pre-trained models and deep reinforcement learning. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, pp. 21314–21328, Red Hook, NY, USA, November 2022. Curran Associates Inc. ISBN 978-1-7138-7108-8.
- Lewkowycz, A., Andreassen, A., Dohan, D., Dyer, E., Michalewski, H., Ramasesh, V., Slone, A., Anil, C., Schlag, I., Gutman-Solo, T., Wu, Y., Neyshabur, B., Gur-Ari, G., and Misra, V. Solving Quantitative Reasoning Problems with Language Models, July 2022. URL <http://arxiv.org/abs/2206.14858>.
- Lu, H., Xie, Z., Wu, Y., Ren, C., Chen, Y., and Wen, Z. OptMATH: A Scalable Bidirectional Data Synthesis Framework for Optimization Modeling, 2025. URL <https://arxiv.org/abs/2502.11102>.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhunoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. Self-Refine: Iterative Refinement with Self-Feedback, May 2023. URL <http://arxiv.org/abs/2303.17651>.
- Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C. L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., and Lowe, R. Training language models to follow instructions with human feedback, March 2022. URL <http://arxiv.org/abs/2203.02155>.
- Parisi, A., Zhao, Y., and Fiedel, N. TALM: Tool Augmented Language Models, May 2022. URL <http://arxiv.org/abs/2205.12255>.
- Pinedo, M. L. Scheduling. *Springer Books*, 2012. URL <https://ideas.repec.org/b/spr/sprbook/978-1-4614-2361-4.html>.
- Ramamonjison, R., Yu, T., Li, R., Li, H., Carenini, G., Ghardar, B., He, S., Mostajabdaveh, M., Banitalebi-Dehkordi, A., Zhou, Z., and Zhang, Y. NL4Opt Competition: Formulating Optimization Problems Based on Their Natural Language Descriptions. In *Proceedings of the NeurIPS 2022 Competitions Track*, pp. 189–203. PMLR, August 2023. URL <https://proceedings.mlr.press/v220/ramamonjison23a.html>.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language Models Can Teach Themselves to Use Tools, February 2023. URL <http://arxiv.org/abs/2302.04761>.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms, 2017. URL <https://arxiv.org/abs/1707.06347>.
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, pp. 8634–8652, Red Hook, NY, USA, December 2023. Curran Associates Inc.
- Toth, P. and Vigo, D. (eds.). *The Vehicle Routing Problem*. Society for Industrial and Applied Mathematics, January 2002. ISBN 978-0-89871-498-2 978-0-89871-851-5. doi: 10.1137/1.9780898718515. URL <http://epubs.siam.org/doi/book/10.1137/1.9780898718515>.
- Turpin, M., Michael, J., Perez, E., and Bowman, S. R. Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, pp. 74952–74965, Red Hook, NY, USA, December 2023. Curran Associates Inc.
- Wei, J., Bosma, M., Zhao, V. Y., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M., and Le, Q. V. Finetuned Language Models Are Zero-Shot Learners, February 2022. URL <http://arxiv.org/abs/2109.01652>.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models, January 2023. URL <http://arxiv.org/abs/2201.11903>.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. Tree of thoughts: Deliberate problem solving with large language models. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, pp. 11809–11822, Red Hook, NY, USA, December 2023. Curran Associates Inc.

A. Training Details

A.1. Model and Hardware Configuration

We use **Qwen2.5-7B-Instruct** as our base model for all experiments. Training is conducted on a single node equipped with **8 NVIDIA H100 GPUs**. We employ the **Verl** framework¹ for distributed reinforcement learning training, which provides efficient actor-critic training with FSDP (Fully Sharded Data Parallel) support.

A.2. Algorithm: PPO with Binary Rewards

Our training uses **Proximal Policy Optimization (PPO)** (Schulman et al., 2017) with binary execution-based rewards. We generate one rollout per prompt and optimize the policy using PPO’s clipped objective, which provides stable updates despite sparse reward signals.

PPO Clipped Objective. The policy loss follows the standard PPO clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E} [\min(r_t(\theta) \cdot A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \cdot A_t)] \quad (6)$$

where $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, and $\epsilon = 0.2$ constrains policy updates.

Reward Assignment and Advantage Computation. We compute a binary terminal reward $r(x, y^{(2)}) \in \{0, 1\}$ by executing only the Turn-2 (audited) program. In implementation, we treat the sequence $(x, y^{(1)}, \rho, y^{(2)})$ as a single trajectory. The terminal reward is first assigned to the last token of the audited response, then summed to obtain a sequence-level score, and finally broadcast to all response tokens across both turns via the advantage computation. Specifically, the advantage is derived from the terminal outcome (using $A = r$ for a nonnegative terminal-only signal), so trajectories with $r = 1$ reinforce both drafting and auditing behaviors that lead to correctness, while trajectories with $r = 0$ do not contribute positive reinforcement. We use ratio clipping for stability; because the advantage is nonnegative in this setting, components designed for negative-advantage handling are not central to our training signal.

We implement this using the Verl framework with PyTorch FSDP for distributed training. Key hyperparameters are summarized in Table 6.

Key Design Decisions. Single Rollout per Prompt. We generate one rollout per prompt, which reduces inference cost during training. We use a batch size of 64 for policy updates.

PPO Clipping. PPO clipping stabilizes training with sparse binary rewards. Without clipping, large policy updates from successful trajectories could destabilize training. The clipped objective ensures conservative updates.

KL Penalty. We disable explicit KL penalty (`use_kl_in_reward=False`) to allow model flexibility in learning optimization code generation, which requires departure from the base model distribution. PPO’s clipping mechanism provides regularization by constraining the ratio between new and old policies to $[1 - \epsilon, 1 + \epsilon]$.

Memory Optimization. Gradient checkpointing and activation offloading accommodate long sequences (up to 19,456 tokens). Sequence packing (`use_remove_padding=True`) reduces padding overhead.

Terminal-Only Verification. We compute reward only from executing the Turn-2 (audited) program. The Turn-1 draft is not executed and the model receives no solver error traces between turns, ensuring that revision behavior must be learned from terminal verification rather than reactive tool feedback.

A.3. Rollout Configuration

We use **SGLang** as the inference engine for rollout generation, providing efficient batched generation with continuous batching. For each prompt, the rollout engine generates $n = 1$ rollout per update. The rollout engine uses 50% GPU memory, balancing throughput with memory available for actor training.

¹<https://github.com/volcengine/verl>

Table 6. Complete training hyperparameters for two-turn DA-RL (Draft→Audit) with terminal-only verification.

Hyperparameter	Value
<i>Model Configuration</i>	
Base model	Qwen2.5-7B-Instruct
<i>Training Configuration</i>	
Learning rate	1×10^{-6}
Optimizer	AdamW (default)
Training batch size	64
PPO mini batch size	64
Micro batch size per GPU	4
Gradient clipping (norm)	1.3
Critic warmup epochs	0
<i>Memory Optimization</i>	
Gradient checkpointing	Enabled
Activation offloading	Enabled
Parameter offloading (FSDP)	False
Optimizer offloading (FSDP)	False
GPU memory utilization	50% (rollout engine)
<i>PPO Clipping Parameters</i>	
Clip ratio (ϵ)	0.2
Clip ratio (lower bound)	0.2
Clip ratio (upper bound)	0.2
Dual-clip parameter (c)	3.0

B. Dataset Statistics and Diagnostics

We evaluate on five diverse optimization benchmarks covering different problem types and difficulty levels. Table 7 summarizes dataset statistics.

Table 7. Dataset statistics with 80/20 train/test splits.

Dataset	Total	Train	Test
IndustryOR	100	80	20
MAMO-Complex	211	169	42
MAMO-Easy	652	522	130
OptMATH-Bench	166	133	33
NL4Opt	230	184	46

Dataset Descriptions. **IndustryOR**: Industrial-grade problems covering resource allocation, scheduling, and logistics with complex logical constraints. **MAMO-Easy**: Standard linear and integer programs. **MAMO-Complex**: Network flow, maximum flow, and combinatorial optimization. **OptMATH-Bench**: Mathematical optimization with detailed numerical specifications. **NL4Opt**: Linear programs from NeurIPS 2022 competition.

Data Splitting and Averaging. To ensure robust evaluation, we perform **three independent random 80/20 splits** (different random seeds) for each dataset. Each split is trained independently, and final reported results are **averaged across all three runs**, mitigating variance from particular train/test partitions.

B.1. Detailed Results Across Three Splits

Table 8 presents the complete pass@1 accuracy for each of the three independent random splits, providing transparency for reviewers to assess variance across data partitions.

Table 8. Detailed pass@1 accuracy (%) across three independent 80/20 splits for DA-RL (Draft-and-Audit RL). Each row corresponds to one dataset, and columns show results for Split 1, Split 2, Split 3, and their mean.

Dataset	Split 1	Split 2	Split 3	Mean
NL4Opt	97.8	97.8	95.7	97.1
MAMO-Easy	91.6	93.1	90.8	91.8
MAMO-Complex	86.0	79.0	88.4	84.5
IndustryOR	40.0	45.0	25.0	36.7
OptMATH-Bench	44.1	23.5	23.5	30.4

B.2. Audit Edit Statistics

Table 9 reports lightweight diff classifications between Turn-1 and Turn-2 programs on the diagnostic logs used in Section 5.5.

Table 9. Audit edit statistics from Turn-1 vs. Turn-2 diffs. Rows report the percentage of analyzed outputs whose diff touches each component.

Edit type	MAMO-Complex	OptMATH-Bench
Any edit	100.0%	100.0%
Constraint	100.0%	100.0%
Domain	100.0%	100.0%
Objective	72.1%	100.0%
Format	100.0%	52.9%

The high edit rate reflects broad re-emission plus targeted semantic repair, while format compliance remains a residual source of execution failures.

C. Reward Function Design

Our reward function evaluates code correctness by sandboxed execution and numerical comparison against ground truth. We use the same notation as the main text: x denotes the word problem, $g(x)$ the ground-truth optimal objective value, $\mathcal{E}(y)$ the sandboxed execution result of program y , and τ the numerical tolerance.

C.1. Execution and Evaluation Pipeline

We execute candidate programs in an isolated environment using **Sandbox Fusion**² (1GB memory limit, 30s timeout). The pipeline is:

1. Extract Python code from the model output (a single markdown code block).
2. Execute in the sandbox and capture stdout.
3. Parse the reported numeric value using a regex pattern such as `answer: (\-?[0-9\.\, ]+)`.
4. Compare the parsed value to the ground truth $g(x)$ using tolerance-based matching.

If extraction, execution, or parsing fails, we treat the outcome as failure and set $\mathcal{E}(y) = \perp$.

C.2. Tolerance-Based Comparison and Binary Verifier

Let $\mathcal{E} : \mathcal{Y} \rightarrow \mathbb{R} \cup \{\perp\}$ denote the execution function. We define a binary terminal verifier for a candidate program y :

$$r(x, y) = \mathbb{K}[\mathcal{E}(y) \neq \perp \wedge \text{Match}(\mathcal{E}(y), g(x); \tau)], \quad (7)$$

²<https://github.com/bytedance/SandboxFusion>

where $\text{Match}(\cdot)$ implements the same tolerance rule as our evaluation protocol. Concretely, we use a dual tolerance with threshold $\tau = 10^{-6}$:

$$\text{Match}(a, b; \tau) = (|a - b| < \tau) \vee \left(\frac{|a - b|}{\max(1, |b|)} < \tau \right). \quad (8)$$

The reward is binary: $r(x, y) = 1$ if the executed value matches the ground truth under this rule, and $r(x, y) = 0$ otherwise. This verifier penalizes both execution failures and *silent modeling failures* (programs that run but yield an incorrect numeric answer).

D. Prompt Template

Each training sample is formatted with a prompt template designed to guide the model toward generating correct and well-structured optimization code. This section presents the complete prompts used in both passes of the Draft-and-Audit interaction.

D.1. Draft Pass: Initial Code Generation Prompt

The draft pass prompt consists of multiple components that guide the model to generate an initial solution. The complete structure is shown below:

```
The following is an optimization problem.

[Problem Description: Natural language description of the optimization problem]

Please think step by step first, then write python code using pycipopt
to solve the above optimization problem and print the final answer.
Your output should follow this template:

[Your thinking process here]

```python
[Your code here, store the final answer in variable `answer`]

print(f"answer: {answer}")
```

Tips:
*****
- Use vtype="C" for divisible quantities: percentages, proportions,
  ratios, fractions, weights, volumes, distances, flow rates,
  continuous time/money (when decimals allowed), and when keywords
  appear: "can be divided", "can be fractional", "any real number",
  "proportion of".
- Use vtype="I" for countable nouns: people, workers, employees,
  customers, students, patients, vehicles, trucks, cars, machines,
  devices, items, products, units, packages, boxes, warehouses, stores,
  facilities, centers, cities, locations, departments, teams, projects,
  tasks, shifts, trips, visits, routes, and when keywords appear:
  "whole number", "indivisible", "how many", "number of".
*****

[Dataset-specific few-shot examples are inserted where applicable; several representative examples are shown in
↪ Appendix\ref{app:fewshot}.]
```

The problem description is dynamically inserted based on the training sample, and dataset-specific few-shot examples are added where they provide domain-relevant guidance.

D.2. Audit Pass: Review and Revision Prompt

After the model generates its initial code in the draft pass, regardless of correctness, it receives the following structured audit instructions:

```
Now enter the checking phase. Review the code you just generated with
respect to the following aspects:
1. Whether the Constraints are set consistently with the problem
   statement, and whether any Constraints are missing.
2. Whether the numerical values of all parameters are consistent with
```

```

the problem description.
3. Whether the variable type settings are reasonable (vtype="C" vs
   vtype="I").
-----
- If and only if you find one or more clear errors in the above
  aspects, please modify the code.
- If you think the errors in the above aspects are not clear (i.e.,
  it is difficult to judge whether modification is needed), do not
  make any changes and directly output the original code in full.
- If there are no clear errors in the above aspects (even if some
  parts could be optimized), do not make any changes and directly
  output the original code in full.
- Regardless of whether modifications are made, you must output the
  code again following this template:
```python
[Your code here, store the final answer in variable `answer`]

print(f"answer: {answer}")
```
- Your new output should be concise, without excessive textual
  analysis, and must contain only a single code block with above form.

```

This audit prompt explicitly directs the model to check for three common error categories: constraint completeness, parameter correctness, and variable type appropriateness. Importantly, the prompt emphasizes that the model should only make changes if clear errors are found, preventing unnecessary modifications that could introduce new bugs.

D.3. Data Construction Pipeline

The preprocessing pipeline (`load_datasets_multiturn.py`) transforms raw JSONL data into two-pass format:

1. Load JSONL with fields: `question`, `answer`, `index`, `ori`
2. Construct prompt: wrap question with instructions, variable type tips, and any dataset-specific few-shot examples
3. Format as single user message; draft response triggers audit prompt, leading to audit response
4. Attach ground truth and metadata for reward computation
5. Split 80/20 with random shuffling (three independent splits for robustness)

Each sample contains: dataset name, prompt (user message with full instructions), reward model config (rule-based with ground truth), and metadata (split, index, answer, interaction config). The Verl framework orchestrates the interaction: presents draft prompt, collects response, invokes interaction handler for audit prompt, collects audit response, and computes reward.

E. Several Few-Shot Examples

We show several few-shot examples used to illustrate representative modeling patterns. We omit dataset-specific examples that are either redundant with the prompt template or not needed for the surrounding discussion.

E.1. MAMO-Complex Examples

```

Example (Maximum Flow):
Embark on a journey through a futuristic transportation network,
connecting 8 bustling metropolises - from a cutting-edge logistics
hub to a far-reaching distribution center. This network is not
ordinary; it's a complex web of superhighways, each capable of
supporting a vast amount of traffic, represented as the capacity
for vehicles (in thousands) that can travel per hour.

[...capacity data for 8 cities...]

Your challenge is to compute the maximum number of vehicles (in
thousands) that can be transported from City 0 (the source) to
City 7 (the destination) per hour, without exceeding the
superhighways' capacities.

Example answer:
This is a maximum flow problem in a network with 8 nodes and

```

```

directed edges with capacity constraints. I should use continuous
variables (vtype="C") for flow since traffic flow rates can be
fractional values.

'''python
from pycscipopt import Model

model = Model("MaxFlow")
capacity = [
    [0, 18, 5, 6, 8, 10, 16, 0],
    [18, 0, 1, 15, 20, 13, 1, 2],
    [18, 16, 0, 19, 15, 4, 9, 10],
    [20, 2, 18, 0, 19, 17, 15, 19],
    [14, 13, 14, 1, 0, 8, 16, 20],
    [0, 17, 18, 10, 17, 0, 16, 6],
    [5, 14, 19, 6, 0, 19, 0, 12],
    [7, 16, 17, 13, 6, 9, 5, 0]
]

flow = {}
for i in range(8):
    for j in range(8):
        if capacity[i][j] > 0:
            flow[i,j] = model.addVar(vtype="C", lb=0,
                                     ub=capacity[i][j],
                                     name=f"flow_{i}_{j}")

total_flow = model.addVar(vtype="C", lb=0, name="total_flow")
model.setObjective(total_flow, "maximize")

for v in range(8):
    inflow = sum(flow.get((i,v), 0) for i in range(8))
    outflow = sum(flow.get((v,j), 0) for j in range(8))
    if v == 0: # Source
        model.addCons(outflow - inflow == total_flow)
    elif v == 7: # Sink
        model.addCons(inflow - outflow == total_flow)
    else: # Intermediate
        model.addCons(inflow == outflow)

model.optimize()
answer = model.getObjVal() if model.getStatus() == "optimal" else 0
print(f"answer: {answer}")
'''

```

E.2. NL4Opt Examples

Example 1 (Integer Variables):
A laundromat can buy two types of washing machines, a top-loading model and a front-loading model. The top-loading model can wash 50 items per day while the front-loading model can wash 75 items per day. The top-loading model costs \$500 to operate per day while the front-loading model costs \$700 per day. If the laundromat needs to wash at least 300 items per day, how many of each type of machine should the laundromat buy to minimize the daily operating cost? Note: machines must be purchased in whole numbers.

Example answer 1:
This is an integer optimization problem where we need to minimize operating cost while meeting washing capacity requirements.

```

'''python
from pycscipopt import Model

model = Model("LaundryMachines")

# Create variables (integer - machines must be whole numbers)
x1 = model.addVar(vtype="I", lb=0, name="x1")
x2 = model.addVar(vtype="I", lb=0, name="x2")

# Set objective (minimize operating cost)
model.setObjective(500*x1 + 700*x2, "minimize")

# Add constraints
model.addCons(50*x1 + 75*x2 >= 300)

# Solve problem
model.optimize()

# Get solution
if model.getStatus() == "optimal":
    answer = model.getObjVal()
else:
    answer = 0

print(f"answer: {answer}")
'''

```

Example 2 (Continuous Variables):
A farmer wants to mix his animal feeds, Feed A and Feed B, in such a way that the mixture will contain a minimum of 30 units of protein and 50 units of fat. Feed A costs \$100 per kilogram and contains 10

units of protein and 20 units of fat per kilogram. Feed B costs \$150 per kilogram and contains 15 units of protein and 25 units of fat per kilogram. How many kilograms of each feed should the farmer use to minimize the cost while meeting the nutritional requirements? Note: feeds can be measured in fractional kilograms.

Example answer 2:
This is a continuous optimization problem where we need to minimize feed cost while meeting nutritional requirements.

```
```python
from pyscipopt import Model

model = Model("AnimalFeed")

Create variables (continuous - feeds can be fractional)
x1 = model.addVar(vtype="C", lb=0, name="x1")
x2 = model.addVar(vtype="C", lb=0, name="x2")

Set objective (minimize cost)
model.setObjective(100*x1 + 150*x2, "minimize")

Add constraints
model.addCons(10*x1 + 15*x2 >= 30)
model.addCons(20*x1 + 25*x2 >= 50)

Solve problem
model.optimize()

Get solution
if model.getStatus() == "optimal":
 answer = model.getObjVal()
else:
 answer = 0

print(f"answer: {answer}")
```
```

E.3. MAMO-Easy Examples

For MAMO-Easy, we found that the problems are relatively straightforward and benefit from minimal scaffolding. Therefore, we provide only the general variable type tips without additional few-shot examples, allowing the model to focus on the problem description itself without overfitting to example patterns.

These examples illustrate the main problem structures, variable types, and modeling patterns used in the prompts, providing concrete templates for generating optimization code.

F. Additional Qualitative Examples

We report six representative qualitative examples where the audit turn makes a substantive correction or cleanup. Each example includes the full draft and the audited code; highlighted lines indicate the main edited regions where applicable.

Example 1. Draft (Turn-1)

```
from pyscipopt import Model

model = Model("TSPModel")

# Define costs in units
costs = [[0, 94, 34, 87],
         [94, 0, 62, 87],
         [34, 62, 0, 97],
         [87, 87, 97, 0]]

# Number of cities
cities = 4

# Define binary variables for each possible route
# 1 if there's a route between cities, 0 otherwise
x = {}
for i in range(cities):
    for j in range(cities):
        if i != j: # We don't need to define routes from a city to itself
            x[i,j] = model.addVar(vtype="B") # Binary variable
        else:
            x[i,j] = 0
    x[j,i] = x[i,j] # Ensure symmetry
```

```
# Objective function: Sum up all costs based on binary variables
objective = sum(x[i, j] * costs[i][j] for i in range(cities) for j in range(cities) if i != j)
model.setObjective(objective, "minimize")

# Ensure the salesperson visits each city exactly once
# We need to visit each city once, so we need to ensure the sum of binary variables for each city is 1
for i in range(cities):
    model.addCons(sum(x[i, j] for j in range(cities) if i != j) == 1) # From city i
    model.addCons(sum(x[j, i] for j in range(cities) if i != j) == 1) # To city i

model.optimize()

# Calculate the total distance
print(f"answer: {model.getObjVal()}")

# To find the route, we can print which routes were taken
route = {}
for i in range(cities):
    for j in range(cities):
        if i != j and x[i, j].x == 1:
            route[i] = j
            break

# Print the route
print("Route:", route)
```

Audit (Turn-2)

```
from pycipopt import Model
model = Model("TSPModel")
costs = [[0, 94, 34, 87], [94, 0, 62, 87], [34, 62, 0, 97], [87, 87, 97, 0]]
cities = 4
x = {}
for i in range(cities):
    for j in range(cities):
        if i != j:
            x[i, j] = model.addVar(vtype="B")
        else:
            x[i, j] = 0
            x[j, i] = x[i, j]
objective = sum(x[i, j] * costs[i][j] for i in range(cities) for j in range(cities) if i != j)
model.setObjective(objective, "minimize")
for i in range(cities):
    model.addCons(sum(x[i, j] for j in range(cities) if i != j) == 1)
    model.addCons(sum(x[j, i] for j in range(cities) if i != j) == 1)
model.optimize()
print(f"answer: {model.getObjVal()}")
```

Example 2. Draft (Turn-1)

```
from pycipopt import Model

model = Model("TSPModel")

# Define costs in units
costs = [[0, 76, 17, 24],
          [76, 0, 84, 11],
          [17, 84, 0, 90],
          [24, 11, 90, 0]]

# Number of cities
cities = 4

# Define binary variables for each possible route
# 1 if there's a route between cities, 0 otherwise
x = {}
for i in range(cities):
    for j in range(cities):
        if i != j: # We don't need to define costs to a city itself
            x[i, j] = model.addVar(vtype="B") # Binary variable
        else:
            x[i, j] = 0
            x[j, i] = x[i, j] # Ensure symmetry

# Objective function: Sum up all costs based on binary variables
objective = sum(x[i, j] * costs[i][j] for i in range(cities) for j in range(cities) if i != j)
```

```

model.setObjective(objective, "minimize")

# Ensure the salesperson visits each city exactly once
# We need to visit each city once, so we need to ensure the sum of binary variables for each city is 1
for i in range(cities):
    model.addCons(sum(x[i, j] for j in range(cities) if i != j) == 1) # From city i
    model.addCons(sum(x[j, i] for j in range(cities) if i != j) == 1) # To city i

model.optimize()

# Calculate the total distance
print(f"answer: {model.getObjVal()}")

# To visualize the path, we can print which routes are taken
path = {}
for i in range(cities):
    for j in range(cities):
        if i != j and x[i, j].x == 1:
            path[i] = j
            break

# Print the path
print("Path:", path)

```

Audit (Turn-2)

```

from pyscipopt import Model
model = Model("TSPModel")
costs = [[0, 76, 17, 24], [76, 0, 84, 11], [17, 84, 0, 90], [24, 11, 90, 0]]
cities = 4
x = {}
for i in range(cities):
    for j in range(cities):
        if i != j:
            x[i, j] = model.addVar(vtype="B")
        else:
            x[i, j] = 0
    x[j, i] = x[i, j]
objective = sum(x[i, j] * costs[i][j] for i in range(cities) for j in range(cities) if i != j)
model.setObjective(objective, "minimize")
for i in range(cities):
    model.addCons(sum(x[i, j] for j in range(cities) if i != j) == 1)
    model.addCons(sum(x[j, i] for j in range(cities) if i != j) == 1)
model.optimize()
print(f"answer: {model.getObjVal()}")

```

Example 3. Draft (Turn-1)

```

from pyscipopt import Model
model = Model("TSPModel")

# Define costs in units
costs = [[0, 13, 87, 76, 50],
          [13, 0, 15, 65, 85],
          [87, 15, 0, 45, 81],
          [76, 65, 45, 0, 80],
          [50, 85, 81, 80, 0]]

# Number of locations
locations = 5

# Define binary variables for each possible route
# 1 if there's a route, 0 if not
x = {}
for i in range(locations):
    for j in range(locations):
        if i != j: # We don't need to define costs to a location itself
            x[i, j] = model.addVar(vtype="B") # Binary variable
        else:
            x[i, j] = 0
    x[j, i] = x[i, j] # Ensure symmetry

# Objective function: Sum up all costs based on binary variables
objective = sum(x[i, j] * costs[i][j] for i in range(locations) for j in range(locations) if i != j)
model.setObjective(objective, "minimize")

```

```

# Ensure the driver visits each location exactly once
# We need to visit 5 locations, so we need to ensure the sum of binary variables for each location is 1
for i in range(locations):
    model.addCons(sum(x[i, j] for j in range(locations) if i != j) == 1) # From location i
    model.addCons(sum(x[j, i] for j in range(locations) if i != j) == 1) # To location i

model.optimize()

# Calculate the total distance
print(f"answer: {model.getObjVal()}")

# To find the path, we can print which routes were taken
path = []
for i in range(locations):
    for j in range(locations):
        if i != j and x[i, j].x == 1:
            path.append((i, j))
print("Path:", path)

```

Audit (Turn-2)

```

from pyscipopt import Model
model = Model("TSPModel")
costs = [[0, 13, 87, 76, 50], [13, 0, 15, 65, 85], [87, 15, 0, 45, 81], [76, 65, 45, 0, 80], [50, 85, 81, 80, 0]]
locations = 5
x = {}
for i in range(locations):
    for j in range(locations):
        if i != j:
            x[i, j] = model.addVar(vtype="B")
        else:
            x[i, j] = 0
    x[j, i] = x[i, j]
objective = sum(x[i, j] * costs[i][j] for i in range(locations) for j in range(locations) if i != j)
model.setObjective(objective, "minimize")
for i in range(locations):
    model.addCons(sum(x[i, j] for j in range(locations) if i != j) == 1)
    model.addCons(sum(x[j, i] for j in range(locations) if i != j) == 1)
model.optimize()
print(f"answer: {model.getObjVal()}")

```

Example 4. Draft (Turn-1)

```

from pyscipopt import Model

model = Model("NutritionOpt")

# Nutritional requirements in grams
requirements = {
    'protein': 60,
    'carb': 146,
    'calories': 1727
}

# Nutritional values per unit (in grams)
values = {
    'chicken': {'protein': 5, 'carb': 13, 'calories': 121},
    'pasta': {'protein': 3, 'carb': 23, 'calories': 274},
    'rice': {'protein': 11, 'carb': 27, 'calories': 251},
    'beans': {'protein': 10, 'carb': 27, 'calories': 259},
    'cheese': {'protein': 4, 'carb': 18, 'calories': 199}
}

# Costs in dollars
costs = {
    'chicken': 7,
    'pasta': 10,
    'rice': 7,
    'beans': 9,
    'cheese': 7
}

# Number of units (assuming you have enough units of each food)
units = 10 # You can adjust this based on how many units you have

```

```

# Define variables with "I" type
food = {}
for food_item in values.keys():
    food[food_item] = model.addVar(vtype="I", name=food_item)

# Objective function: Sum up the costs based on the units consumed
objective = sum(food[food_item] * costs[food_item] for food_item in values.keys())
model.setObjective(objective, "minimize")

# Ensure we meet the requirements
# Ensure we consume at least the required nutrients
model.addCons(sum(food[food_item] * values[food_item]['protein'] for food_item in values.keys()) >=
    requirements['protein'])
model.addCons(sum(food[food_item] * values[food_item]['carb'] for food_item in values.keys()) >=
    requirements['carb'])
model.addCons(sum(food[food_item] * values[food_item]['calories'] for food_item in values.keys()) >=
    requirements['calories'])

model.optimize()

# Calculate the total cost
print(f"answer: {model.getObjVal()}")

# Print how many units of each food are consumed
for food_item in values.keys():
    if food[food_item].x > 0:
        print(f"{food_item}: {food[food_item].x}")

```

Audit (Turn-2)

```

from pycipopt import Model
model = Model("NutritionOpt")
requirements = {'protein': 60, 'carb': 146, 'calories': 1727}
values = {'chicken': {'protein': 5, 'carb': 13, 'calories': 121}, 'pasta': {'protein': 3, 'carb': 23, 'calories':
    274}, 'rice': {'protein': 11, 'carb': 27, 'calories': 251}, 'beans': {'protein': 10, 'carb': 27, 'calories':
    259}, 'cheese': {'protein': 4, 'carb': 18, 'calories': 199}}
costs = {'chicken': 7, 'pasta': 10, 'rice': 7, 'beans': 9, 'cheese': 7}
units = 10
food = {}
for food_item in values.keys(): food[food_item] = model.addVar(vtype="I", name=food_item)
objective = sum(food[food_item] * costs[food_item] for food_item in values.keys())
model.setObjective(objective, "minimize")
model.addCons(sum(food[food_item] * values[food_item]['protein'] for food_item in values.keys()) >=
    requirements['protein'])
model.addCons(sum(food[food_item] * values[food_item]['carb'] for food_item in values.keys()) >=
    requirements['carb'])
model.addCons(sum(food[food_item] * values[food_item]['calories'] for food_item in values.keys()) >=
    requirements['calories'])
model.optimize()
print(f"answer: {model.getObjVal()}")

```

Example 5. Draft (Turn-1)

```

from pycipopt import Model

model = Model("TSPModel")

# Define costs in units
costs = [[0, 62, 51, 50],
         [62, 0, 42, 69],
         [51, 42, 0, 11],
         [50, 69, 11, 0]]

# Number of locations
locations = 4

# Define binary variables for routes
# 1 if there's a route, 0 if not
x = {}
for i in range(locations):
    for j in range(locations):
        if i != j: # We don't need to define costs to a location itself
            x[i,j] = model.addVar(vtype="B") # Binary variable
        else:
            x[i,j] = 0

```

```

x[j,i] = x[i,j] # Ensure symmetry

# Objective function: Sum up all costs based on binary variables
objective = sum(x[i, j] * costs[i][j] for i in range(locations) for j in range(locations) if i != j)
model.setObjective(objective, "minimize")

# Ensure the courier visits each location exactly once
# We need to visit each location once, so we need to visit 1 location from each starting point
for i in range(locations):
    model.addCons(sum(x[i, j] for j in range(locations) if i != j) == 1) # From location i
    model.addCons(sum(x[j, i] for j in range(locations) if i != j) == 1) # To location i

model.optimize()

# Calculate the total distance
print(f"answer: {model.getObjVal()}")

# To find the path, we can print the routes
path = {}
for i in range(locations):
    for j in range(locations):
        if i != j and x[i, j].x == 1:
            path[i] = j
            break

# Print the path starting from any location (assuming we start from location 0)
start = 0
print(f"Path starting from location {start}: {start}", end=" -> ")
current = start
while path:
    next_loc = path[current]
    print(f"{next_loc}", end=" -> ")
    current = next_loc
print("start")

```

Audit (Turn-2)

```

from pyscipopt import Model
model = Model("TSPModel")
costs = [[0, 62, 51, 50], [62, 0, 42, 69], [51, 42, 0, 11], [50, 69, 11, 0]]
locations = 4
x = {}
for i in range(locations):
    for j in range(locations):
        if i != j:
            x[i, j] = model.addVar(vtype="B")
        else:
            x[i, j] = 0
    x[j, i] = x[i, j]
objective = sum(x[i, j] * costs[i][j] for i in range(locations) for j in range(locations) if i != j)
model.setObjective(objective, "minimize")
for i in range(locations):
    model.addCons(sum(x[i, j] for j in range(locations) if i != j) == 1)
    model.addCons(sum(x[j, i] for j in range(locations) if i != j) == 1)
model.optimize()
print(f"answer: {model.getObjVal()}")

```

Example 6. Draft (Turn-1)

```

from pyscipopt import Model

# Create the model
model = Model("facility_selection")

# Define nodes and distances
nodes = [0, 1, 2, 3, 4]
distances = {
    (0, 1): 41, (0, 2): 31, (0, 3): 43, (0, 4): 10,
    (1, 2): 34, (1, 3): 46, (1, 4): 12,
    (2, 3): 41, (2, 4): 24, (3, 4): 23
}

# Define binary variable for selecting each node
s = {node: model.addVar(vtype="B", lb=0, name=f"select_{node}") for node in nodes}

# Define binary variable for minimum distance

```

```

m = model.addVar(vtype="C", lb=0, name="min_distance")

# Ensure exactly two nodes are selected
model.addCons(sum(s[node] for node in nodes) == 2)

# Ensure minimum distance between any two selected nodes
for (i, j), d in distances.items():
    if i != j:
        p = model.addVar(vtype="B", lb=0, name=f"pair_{i}_{j}")
        model.addCons(d >= m * p)
        model.addCons(p <= s[i])
        model.addCons(p <= s[j])
        model.addCons(p >= s[i] + s[j] - 1)
        model.addCons(d >= m * p + 1000000 * (1 - p))

# Set the objective to maximize the minimum distance
model.setObjective(m, "maximize")

# Solve the model
model.optimize()

# Get the optimal solution
if model.getStatus() == "optimal":
    answer = model.getObjVal()
else:
    answer = 0

print(f"answer: {answer}")

```

Audit (Turn-2)

```

from pycipopt import Model

# Create the model
model = Model("facility_selection")

# Define nodes and distances
nodes = [0, 1, 2, 3, 4]
distances = {
    (0, 1): 41, (0, 2): 31, (0, 3): 43, (0, 4): 10,
    (1, 2): 34, (1, 3): 46, (1, 4): 12,
    (2, 3): 41, (2, 4): 24, (3, 4): 23
}

# Define binary variable for selecting each node
s = {node: model.addVar(vtype="B", lb=0, name=f"select_{node}") for node in nodes}

# Ensure exactly two nodes are selected
model.addCons(sum(s[node] for node in nodes) == 2)

# Define binary variable for minimum distance
m = model.addVar(vtype="C", lb=0, name="min_distance")

# Ensure minimum distance between any two selected nodes
for (i, j), d in distances.items():
    if i != j:
        p = model.addVar(vtype="B", lb=0, name=f"pair_{i}_{j}")
        model.addCons(d >= m * p)
        model.addCons(p <= s[i])
        model.addCons(p <= s[j])
        model.addCons(p >= s[i] + s[j] - 1)

# Set the objective to maximize the minimum distance
model.setObjective(m, "maximize")

# Solve the model
model.optimize()

# Get the optimal solution
if model.getStatus() == "optimal":
    answer = model.getObjVal()
else:
    answer = 0

print(f"answer: {answer}")

```